Homework 2: TDD Practice with Calc

Final Version of Calc.py

Welcome | output | CalcTest.py | Calc.py

Calc.py > Calculator > divide

```python
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return float(a) / float(b)
```

Final Version of CalcTest.py

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
import unittest
from Calc import Calculator  # The class we are going to implement

class TestCalculator(unittest.TestCase):
    def test_add(self):
        calc = Calculator()
        result = calc.add(2, 3)
        self.assertEqual(result, 5)  # Expect 2 + 3 = 5

    def test_subtract(self):
        calc = Calculator()
        self.assertEqual(calc.subtract(10, 4), 6)    # Expect 10 - 4 = 6
        self.assertEqual(calc.subtract(4, 4), 0)     # Expect 4 - 4 = 0
        self.assertEqual(calc.subtract(4, 10), -6)   # Expect 4 - 10 = -6

    def test_multiply(self):
        calc = Calculator()
        self.assertEqual(calc.multiply(5, 5), 25)    # Expect 5 * 5 = 25
        self.assertEqual(calc.multiply(5, 0), 0)     # Expect 5 * 0 = 0
        self.assertEqual(calc.multiply(5, -5), -25)  # Expect 5 * -5 = -25
        self.assertEqual(calc.multiply(-5, -5), 25)  # Expect -5 * -5 = 25

    def test_divide(self):
        calc = Calculator()
        self.assertEqual(calc.divide(9, 3), 3.0)    # Expect 9 / 3 = 3.0
        self.assertEqual(calc.divide(9, -3), -3.0)   # Expect 9 / -3 = -3.0
        self.assertEqual(calc.divide(-9, -3), 3.0)   # Expect -9 / -3 = 3.0
        self.assertRaises(ValueError, calc.divide, 9, 0) # Expect 9 / 0 = ValueError

if __name__ == "__main__":
    unittest.main()
```

All Tests Passing

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s
```

Starting Point

The original `Calc.py` contained a single `add()` method. All new functionality was added using Test-Driven Development, following the **Red** → **Green** → **Refactor** cycle.

TDD Cycle 1: subtract()

Red: A failing test `test_subtract` was written in `CalcTest.py` before any implementation existed. The test covered three scenarios upfront: a positive result `(10, 4)` expecting `6`, a zero result `(4, 4)` expecting `0`, and

a negative result `(4, 10)` expecting `-6`. Running the test produced `AttributeError: 'Calculator' object has no attribute 'subtract'`, confirming the test correctly failed.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
class TestCalculator(unittest.TestCase):
    def test_subtract(self):
        calc = Calculator()
        result = calc.subtract(10, 4)
        self.assertEqual(result, 6)  # Expect 10 - 4 = 6
        result = calc.subtract(4, 4)
        self.assertEqual(result, 0)  # Expect 4 - 4 = 0
        result = calc.subtract(4, 10)
        self.assertEqual(result, -6)  # Expect 4 - 10 = -6



if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
.E
======================================================================
ERROR: test_subtract (__main__.TestCalculator.test_subtract)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "C:\Users\l3303\Downloads\TDD\CalcTest.py", line 12, in test_subtract
    result = calc.subtract(10, 4)
             ^^^^^^^^^^^^^
AttributeError: 'Calculator' object has no attribute 'subtract'

----------------------------------------------------------------------
Ran 2 tests in 0.001s

FAILED (errors=1)
(base) PS C:\Users\l3303\Downloads\TDD>
```

Green: The `subtract()` method was added to `Calc.py` with the implementation `return a - b`. All three assertions passed.

Welcome | output | CalcTest.py | Calc.py

Calc.py > ...

```python
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
(base) PS C:\Users\l3303\Downloads\TDD>
```

Refactor: No changes were made to the implementation logic. The repeated `result =` variable assignments in `test_subtract` were removed, passing values directly into `assertEqual()` to improve code readability. All tests continued to pass.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
class TestCalculator(unittest.TestCase):
    def test_subtract(self):
        calc = Calculator()
        self.assertEqual(calc.subtract(10, 4), 6)    # Expect 10 - 4 = 6
        self.assertEqual(calc.subtract(4, 4), 0)     # Expect 4 - 4 = 0
        self.assertEqual(calc.subtract(4, 10), -6)   # Expect 4 - 10 = -6



if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
..
----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
(base) PS C:\Users\l3303\Downloads\TDD>
```

TDD Cycle 2: multiply()

Red: A failing test `test_multiply` was written covering four scenarios upfront: two positives `(5, 5)` expecting `25`, multiply by zero `(5, 0)` expecting `0`, mixed signs `(5, -5)` expecting `-25`, and two negatives `(-5, -5)` expecting `25`. Running the test produced `AttributeError: 'Calculator' object has no attribute 'multiply'`, confirming the correct failure.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > TestCalculator > test_multiply

```python
class TestCalculator(unittest.TestCase):
    def test_multiply(self):
        calc = Calculator()
        result = calc.multiply(5, 5)
        self.assertEqual(result, 25)  # Expect 5 * 5 = 25
        result = calc.multiply(5, 0)
        self.assertEqual(result, 0)  # Expect 5 * 0 = 0
        result = calc.multiply(5, -5)
        self.assertEqual(result, -25)  # Expect 5 * -5 = -25
        result = calc.multiply(-5, -5)
        self.assertEqual(result, 25)  # Expect -5 * -5 = 25

if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
.E.
======================================================================
ERROR: test_multiply (__main__.TestCalculator.test_multiply)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "C:\Users\l3303\Downloads\TDD\CalcTest.py", line 20, in test_multiply
    result = calc.multiply(5, 5)
             ^^^^^^^^^^^^^^
AttributeError: 'Calculator' object has no attribute 'multiply'

----------------------------------------------------------------------
Ran 3 tests in 0.001s

FAILED (errors=1)
(base) PS C:\Users\l3303\Downloads\TDD>
```

Green: The `multiply()` method was added to `Calc.py` with the implementation `return a * b`. All four assertions passed.

Welcome | output | CalcTest.py | Calc.py

Calc.py > Calculator > multiply

```python
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
(base) PS C:\Users\l3303\Downloads\TDD>
```

Refactor: No changes were made to the implementation logic. The repeated `result =` variable assignments in `test_multiply` were removed, passing values directly into `assertEqual()` to improve code readability. All tests continued to pass.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
class TestCalculator(unittest.TestCase):
    def test_multiply(self):
        calc = Calculator()
        result = calc.multiply(5, 5)
        self.assertEqual(calc.multiply(5, 5), 25)    # Expect 5 * 5 = 25
        self.assertEqual(calc.multiply(5, 0), 0)     # Expect 5 * 0 = 0
        self.assertEqual(calc.multiply(5, -5), -25)  # Expect 5 * -5 = -25
        self.assertEqual(calc.multiply(-5, -5), 25)  # Expect -5 * -5 = 25

if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
(base) PS C:\Users\l3303\Downloads\TDD>
```

TDD Cycle 3: divide()

Before writing any tests, two design decisions were made and encoded into the tests:

- `divide()` should return a **float**, not an integer.
- Division by zero should raise a `ValueError`.

Cycle 3a: Basic Division

Red: A failing test `test_divide` was written calling `calc.divide(9, 3)` and expecting `3.0`. Running the test produced `AttributeError: 'Calculator' object has no attribute 'divide'`, confirming the correct failure.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
class TestCalculator(unittest.TestCase):
    def test_divide(self):
        calc = Calculator()
        result = calc.divide(9,3)
        self.assertEqual(result, 3.0) # Expect 9 / 3 = 3.0


if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
.E..
======================================================================
ERROR: test_divide (__main__.TestCalculator.test_divide)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "C:\Users\l3303\Downloads\TDD\CalcTest.py", line 30, in test_divide
    result = calc.divide(9,3)
             ^^^^^^^^^^^^
AttributeError: 'Calculator' object has no attribute 'divide'

----------------------------------------------------------------------
Ran 4 tests in 0.001s

FAILED (errors=1)
```

Green: The `divide()` method was added to `Calc.py` with the implementation `return float(a) / float(b)`. The test passed.

Welcome | output | CalcTest.py | Calc.py

Calc.py > Calculator > divide

```python
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        return float(a) / float(b)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

Refactor: No changes were made to the implementation logic. The `result =` variable assignment in `test_divide` was removed, passing the value directly into `assertEqual()` to improve code readability. All tests continued to pass.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
class TestCalculator(unittest.TestCase):

    def test_divide(self):
        calc = Calculator()
        self.assertEqual(calc.divide(9, 3), 3.0)    # Expect 9 / 3 = 3.0


if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

Cycle 3b: Division by Zero

Red: Additional test cases were added to `test_divide`: negative divisor `(9, -3)` expecting `-3.0`, two negatives `(-9, -3)` expecting `3.0`, and division by zero using `assertRaises(ValueError, calc.divide, 9, 0)`. Running the tests produced `ZeroDivisionError` instead of the expected `ValueError`, confirming the correct failure.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > ...

```python
class TestCalculator(unittest.TestCase):
    def test_divide(self):
        calc = Calculator()
        self.assertEqual(calc.divide(9, 3), 3.0)    # Expect 9 / 3 = 3.0
        result = calc.divide(9,-3)
        self.assertEqual(result, -3.0) # Expect 9 / -3 = -3.0
        result = calc.divide(-9,-3)
        self.assertEqual(result, 3.0) # Expect -9 / -3 = 3.0
        self.assertRaises(ValueError, calc.divide, 9, 0) # Expect 9 / 0 = ValueError

if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
.E..
======================================================================
ERROR: test_divide (__main__.TestCalculator.test_divide)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "C:\Users\l3303\Downloads\TDD\CalcTest.py", line 30, in test_divide
    self.assertRaises(ValueError, calc.divide, 9, 0) # Expect 9 / 0 = ValueError
    ~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\l3303\anaconda3\Lib\unittest\case.py", line 795, in assertRaises
    return context.handle('assertRaises', args, kwargs)
           ~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\l3303\anaconda3\Lib\unittest\case.py", line 238, in handle
    callable_obj(*args, **kwargs)
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^
  File "C:\Users\l3303\Downloads\TDD\Calc.py", line 12, in divide
    return float(a) / float(b)
           ~~~~~~~~~^~~~~~~~~~
ZeroDivisionError: float division by zero

----------------------------------------------------------------------
Ran 4 tests in 0.001s

FAILED (errors=1)
```

Green: A guard clause was added to `divide()`: `if b == 0: raise ValueError("Cannot divide by zero")`. All 4 tests passed.

Welcome | output | CalcTest.py | Calc.py

Calc.py > Calculator > multiply

```python
class Calculator:
    def divide(self, a, b):
        if b == 0:
            raise ValueError("Cannot divide by zero")
        return float(a) / float(b)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

Refactor: No changes were made to the implementation logic. The remaining `result =` variable assignments in `test_divide` were removed, passing values directly into `assertEqual()` to improve code readability. All 4 tests continued to pass.

Welcome | output | CalcTest.py | Calc.py

CalcTest.py > TestCalculator > test_divide

```python
class TestCalculator(unittest.TestCase):
    def test_divide(self):
        calc = Calculator()
        self.assertEqual(calc.divide(9, 3), 3.0)    # Expect 9 / 3 = 3.0
        self.assertEqual(calc.divide(9, -3), -3.0)   # Expect 9 / -3 = -3.0
        self.assertEqual(calc.divide(-9, -3), 3.0)   # Expect -9 / -3 = 3.0
        self.assertRaises(ValueError, calc.divide, 9, 0) # Expect 9 / 0 = ValueError

if __name__ == "__main__":
    unittest.main()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(base) PS C:\Users\l3303\Downloads\TDD> python .\CalcTest.py
....
----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```